MediCare+: Smart Digital Healthcare & Clinical Symptom Triage Portal

A Report submitted under Project-Based Learning
In Partial Fulfillment of the Course Requirements for
"Web Technologies (22IT104001)"

Submitted By

SRAVANTHI	**(24102A040629)**
REKHA	**(24102A040623)**
VYSHNAVI	**(24102A040619)**

Under the Guidance of

PATAN FERIDOZ KHAN M.Tech(Ph.D)

Assistant Professor

Department of CSE

Department of Computer Science and Engineering

School of Computing

MOHAN BABU UNIVERSITY

Sree Sainath Nagar, Tirupati – 517 102

2025-2026

MOHAN BABU UNIVERSITY

Vision

To be a globally respected institution with an innovative and entrepreneurial culture that offers transformative education to advance sustainability and societal good.

Mission

- Develop industry-focused professionals with a global perspective.
- Offer academic programs that provide transformative learning experience founded on the spirit of curiosity, innovation, and integrity.
- Create confluence of research, innovation, and ideation to bring about sustainable and socially relevant enterprises.
- Uphold high standards of professional ethics leading to harmonious relationship with environment and society.

SCHOOL OF COMPUTING

Vision

To lead the advancement of computer science research and education that has real-world impact and to push the frontiers of innovation in the field.

Mission

- Instil within our students fundamental computing knowledge, a broad set of skills, and an inquisitive attitude to create innovative solutions to serve industry and community.
- Provide an experience par excellence with our state-of-the-art research, innovation, and incubation ecosystem to realise our learners' fullest potential.
- Impart continued education and research support to working professionals in the computing domain to enhance their expertise in the cutting-edge technologies.
- Inculcate among the computing engineers of tomorrow with a spirit to solve societal challenges.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision

To become a Centre of Excellence in Computer Science and its emerging areas by imparting high quality education through teaching, training and research.

Mission

- Imparting quality education in Computer Science and Engineering and emerging areas of IT industry by disseminating knowledge through contemporary curriculum, competent faculty and effective teaching-learning methodologies.
- Nurture research, innovation and entrepreneurial skills among faculty and students to contribute to the needs of industry and society.
- Inculcate professional attitude, ethical and social responsibilities for prospective and promising engineering profession.
- Encourage students to engage in life-long learning by creating awareness of the contemporary developments in Computer Science and Engineering and its emerging areas.

B.Tech. Computer Science and Engineering

PROGRAM EDUCATIONAL OBJECTIVES

After few years of graduation, the graduates of B.Tech. CSE will be:

- **PEO1.** Pursuing higher studies in core, specialized or allied areas of Computer Science, or Management.
- **PEO2.** Employed in reputed Computer and I.T organizations or Government to have a globally competent professional career in Computer Science and Engineering domain or be successful Entrepreneurs.
- **PEO3.** Able to demonstrate effective communication, engage in teamwork, exhibit leadership skills and ethical attitude, and achieve professional advancement through continuing education.

PROGRAM OUTCOMES

On successful completion of the Program, the graduates of B.Tech. CSE Program will be able to:

- **PO1. Engineering Knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- **PO2. Problem Analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- **PO3. Design/Development of Solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- **PO4. Conduct Investigations of Complex Problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- **PO5. Modern Tool Usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- **PO6. The Engineer and Society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- **PO7. Environment and Sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

- **PO8. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- **PO9. Individual and Team Work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- **PO10. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- **PO11. Project Management and Finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- **PO12. Life-long Learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES

On successful completion of the Program, the graduates of B. Tech. (CSE) program will be able to:

- **PSO1.** Apply knowledge of computer science engineering, Use modern tools, techniques and technologies for efficient design and development of computer-based systems for complex engineering problems.
- **PSO2.** Design and deploy networked systems using standards and principles, evaluate security measures for complex networks, apply procedures and tools to solve networking issues.
- **PSO3.** Develop intelligent systems by applying adaptive algorithms and methodologies for solving problems from inter-disciplinary domains.
- **PSO4.** Apply suitable models, tools and techniques to perform data analytics for effective decision making.

Course Code	Course Title	L	T	P	S	C
22IT104001	**WEB TECHNOLOGIES**	3	-	2	4	5

COURSE OUTCOMES:

After successful completion of this course, the students will be able to:

- **CO1.** Demonstrate knowledge on web page design elements, dynamic content and database connection.
- **CO2.** Analyze user requirements to develop web applications.
- **CO3.** Design client-server applications using web technologies.
- **CO4.** Demonstrate problem solving skills to develop enterprise web applications.

- **CO5.** Apply HTML, CSS, JavaScript, JQuery, Bootstrap and PHP technologies for device independent web application development.
- **CO6.** Apply web technologies to develop interactive, dynamic and scalable web applications for societal needs.

CO-PO-PSO Mapping Table:

Course Outcomes	Program Outcomes												Program Specific Outcomes			
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3	P
CO1	3	3	2	-	-	-	-	-	-	-	-	-	3	2	3	-
CO2	3	3	3	2	2	-	-	-	-	-	-	-	3	2	3	-
CO3	3	3	3	2	2	-	-	-	-	-	-	-	3	2	3	-
CO4	3	3	3	2	2	-	-	-	-	-	-	-	2	2	3	-
CO5	3	2	2	2	2	3	-	-	-	-	-	-	2	2	3	-
CO6	-	-	-	2	-	-	-	-	3	3	-	-	-	-	-	-
Course Correlation Mapping	**3**	**3**	**3**	**2**	**2**	**-**	**-**	**-**	**3**	**3**	**-**	**-**	**3**	**2**	**3**	**-**

Correlation Levels: 3: High; 2: Medium; 1: Low

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Project Entitled **"MediCare+: Smart Digital Healthcare & Clinical Symptom Triage Portal"** Submitted By:

SRAVANTHI	**(24102A040629)**
REKHA	**(24102A040623)**
VYSHNAVI	**(24102A040619)**

is the work submitted under Project-Based Learning in Partial Fulfillment of the Course Requirements for **"Web Technologies (22IT104001)"** during **2025-2026**.

Supervisor:

PATAN FERIDOZ KHAN M.Tech(Ph.D)
Assistant Professor
Department of CSE
School of Computing
Mohan Babu University
Tirupati.

Head:

Dr. G. Sunitha
Professor & Head
Department of CSE
School of Computing
Mohan Babu University
Tirupati.

ACKNOWLEDGEMENTS

First and foremost, I extend my sincere thanks to **Dr. M. Mohan Babu**, Chancellor, for his unwavering support and vision that fosters academic excellence within the institution.

My gratitude also goes to **Mr. Manchu Vishnu**, Pro-Chancellor, for creating an environment that promotes creativity and for his encouragement and commitment to student success.

I am deeply appreciative of **Prof. Nagaraj Ramrao**, Vice Chancellor, whose leadership has created an environment conducive to learning and innovation.

I would like to thank **Dr. K. Saradhi**, Registrar, for his support in creating an environment conducive to academic success.

I am also grateful to **Dr. G. Sunitha**, Head of the Department of Computer Science and Engineering, for her valuable insights and support.

Finally, I would like to express my deepest appreciation to my project supervisor, **PATAN FERIDOZ KHAN**, Assistant Professor, Department of Computer Science and Engineering for continuous guidance, encouragement, and expertise throughout this project.

Thank you all for your support and encouragement.

Table of Contents

Chapter No.	Title	Page No.
	Abstract	1
1	**Introduction**	2
1.1	Problem Statement	3
1.2	Importance of the Problem	3
1.3	Objectives	4
1.4	Scope of the Project	5
2	**System Design**	6
2.1	Architecture Diagram	7
2.2	Module Descriptions	9
2.3	Database Design	11
3	**Implementation**	12
3.1	Tools and Technologies Used	13
3.2	Front-End Development	15
3.3	Back-End Development	17
3.4	Integration	18
4	**Testing, Results and Discussion**	21
4.1	Test Cases	22
4.2	Testing Methods	24
4.3	Output Screenshots	25
4.4	Analysis of Results	28
5	**Conclusion**	29
5.1	Summary of Findings	30
5.2	Future Enhancements	31
6	**Appendix**	32
6.1	Code Snippets	34

ABSTRACT

The project **“MediCare+: Smart Digital Healthcare & Clinical Symptom Triage Portal”** presents an interactive, responsive web-based platform designed to address primary healthcare accessibility barriers and streamline outpatient department (OPD) hospital consultations. The website provides a single, easy-to-use interface through which patients can learn about health services, evaluate acute physical symptoms using a clinical triage algorithm, discover verified medical specialists, schedule consultation visits, generate printable hospital OPD passes, and monitor daily physiological vitals.

The system is developed using **HTML5, CSS3, and JavaScript**. HTML is used to structure the web pages, CSS is used to create a responsive and visually attractive interface that adapts to mobile phones, tablets, and desktop screens with custom themes and dark mode, and JavaScript is used to implement interactive features, mathematical algorithms, and dynamic content. The project follows a modular single-page website approach in which different health modules are connected through a sticky navigation bar.

The major features of the system include the **Home Page, Clinical Symptom Triage Checker, Doctor Directory with Live Search, OPD Appointment Booking Desk, Official Hospital Pass Generator, and Personal Health Vitals Calculators** (BMI, Hydration, and Caloric TDEE). The symptom triage engine utilizes a multi-step guided wizard that weighs symptom severity, duration, and patient pain scales to classify urgency into *Low*, *Moderate*, or *Urgent* care pathways while automatically recommending the relevant medical specialty.

Appointments booked by patients are dynamically recorded in the browser's `localStorage`, allowing persistent record keeping without server setup. Furthermore, the portal includes an automated Hospital Verification Slip Generator formatted for official printing via `@media print` directives. The current implementation is a functional client-side prototype using sample data and client-side JavaScript. Future versions can be enhanced with a cloud database, tele-consultation video integration, online payments, SMS/WhatsApp notifications, and map integration.

CHAPTER 1:
INTRODUCTION

1.INTRODUCTION

1.1 Problem Statement

Healthcare clinics, nursing homes, and outpatient departments (OPDs) form the backbone of community health services in every town and city. These centers serve neighbourhoods and provide crucial diagnostic care. However, a large number of local healthcare setups still depend entirely on manual counter registrations, physical paper tokens, and phone calls to coordinate patient appointments.

In today's digital world, patients search for healthcare providers, doctors, and clinic timings on their mobile phones before heading out. When a local hospital clinic does not have a digital portal, patients cannot easily verify doctor availability, consultation fees, specialist degrees, or working hours. As a result, clinics experience mismanaged queues and long waiting hours, while patients lose critical time during acute health emergencies.

The **MediCare+ Smart Digital Healthcare** project addresses these problems by providing a fast, attractive, mobile-friendly and accessible portal that any local clinic or healthcare center can use to enhance its digital presence. The website presents clinical credentials, doctor profiles with consultation fees, an interactive symptom triage questionnaire, live hospital status, and a simple way to book appointments and generate verifiable passes.

The project interface itself describes its central purpose as helping patients evaluate symptoms, discover verified medical specialists, schedule visits, and access personal wellness tools from any device.

1.2 Importance of the Problem

A strong digital presence in healthcare is no longer a luxury. For a community clinic, it is the digital equivalent of an immediate triage desk. When patients cannot find reliable medical information or assess the urgency of their symptoms, they risk ignoring red-flag conditions or crowding emergency rooms unnecessarily.

One major issue is the lack of discoverability. Patients who search for a nearby cardiologist, pediatrician, or dermatologist on their phone expect to see qualification credentials, consulting fees, and available time slots within seconds. The MediCare+ project addresses this by displaying all of this information in a clean, well-organized website that loads quickly even on slow mobile connections.

Another important issue is trust. A professional healthcare portal with verified doctor profiles, patient ratings, and transparent fees makes patients feel confident about visiting the clinic. The Doctor Directory and Testimonials modules of the project are designed to build this trust.

The project also considers the problem of convenience. Patients often do not want to wait in physical registration queues just to secure a basic consultation slot. The Appointment module allows selecting exact consultation times and instantly generating a printable verification pass.

Similarly, the cost of custom software development is often beyond the budget of small local clinics. A reusable, zero-dependency front-end web template built with standard technologies allows healthcare providers to establish an online presence at minimal cost.

Therefore, the project is important because it combines information, triage guidance, doctor profiles, scheduling, and health vitals into a single unified platform. It demonstrates how web technologies can solve real societal healthcare needs, which is the central aim of the Web Technologies course.

1.3 Objectives

The main objectives of the MediCare+ Project are:

1. **To develop a user-friendly healthcare portal:** To create an attractive and simple web interface through which patients can access all important clinic and consultation information.
2. **To implement an interactive symptom triage checker:** To provide an algorithmic questionnaire that categorizes symptoms into Low, Moderate, or Urgent care pathways.

3. **To present medical specialists clearly:** To provide a Doctor Directory with qualifications, experience, fees, and specialty filters so patients can quickly find the right physician.
4. **To build patient trust:** To provide transparent consultation fees, patient review ratings, and hospital credentials.
5. **To simplify appointment booking:** To provide a dynamic slot picker, form validation, and instant appointment token generation with browser persistence.
6. **To create a reusable and customisable template:** To separate content from design so the website can be adapted for general hospitals, dental clinics, or diagnostic centers.
7. **To implement interactive web technologies:** To demonstrate practical usage of HTML, CSS, and JavaScript for developing an interactive web application.
8. **To develop a responsive interface:** To ensure that the website adapts to different screen sizes and provides a usable experience on mobile phones, tablets and desktops.
9. **To keep the website fast, accessible and search-friendly:** To use semantic HTML, descriptive meta tags, and accessible controls with dark mode support.

1.4 Scope of the Project

The scope of the MediCare+ project covers several aspects of a modern healthcare facility's online presence and patient engagement.

The Home section introduces the healthcare portal with a hero banner, emergency helpline numbers (108/112), call-to-action buttons, key statistics, and verified patient reviews.

The Symptom Checker section allows patients to enter age, gender, and pre-existing medical conditions, select from categorized symptom chips, and calibrate duration and pain severity. The algorithm outputs a triage level and directs the user to the relevant medical department.

The Doctor Directory displays verified specialists with photo avatars, clinical degrees, ratings, and consultation fees, complete with real-time text searching and department filtering.

The Appointments module provides an online booking desk that validates user inputs, prevents past-date selection, offers dynamic time slots, and records bookings into browser `localStorage`. It generates an official Hospital Verification Slip with a simulated barcode for physical printing.

The Health Vitals module provides instant calculators for Body Mass Index (BMI) with a colored spectrum gauge, Daily Water Intake, and Mifflin-St Jeor Caloric Needs.

The current project is primarily a front-end interactive prototype. Its doctor data, symptoms catalog, appointments, and vitals logic are maintained using client-side JavaScript. The project does not include online payments, user authentication, or a server-side database, which are listed as future enhancements.

CHAPTER 2:
SYSTEM DESIGN

2. SYSTEM DESIGN

The system design of the MediCare+ project describes the overall structure, components, modules, and interaction flow of the web application. The main purpose of the system design is to organize the different healthcare functionalities into a single interactive and easy-to-navigate website.

The website is designed as a client-side web application using HTML, CSS, and JavaScript. HTML provides the structural elements of the application, CSS provides the visual presentation, responsive layout, and dark mode, and JavaScript manages dynamic content, triage algorithms, and user interactions.

The application follows a modular design approach. Instead of placing all information in one unstructured page, the website divides the functionality into distinct modules: Home, Symptom Checker, Doctor Directory, Appointments Desk, and Health Vitals.

The main navigation provides direct access to these modules. On small screens, the menu collapses into a responsive hamburger button.

2.1 Architecture Diagram

The architecture of the MediCare+ project consists of three major technical layers:

1. **Presentation Layer:** HTML and CSS define structure, styling, theme variables, cards, buttons, and responsive layouts.
2. **Application Logic Layer:** JavaScript processes triage scoring, live filtering, dynamic time slots, form validation, and modal triggers.
3. **Client-Side Data Layer:** The prototype stores sample information using JavaScript arrays and browser `localStorage`.

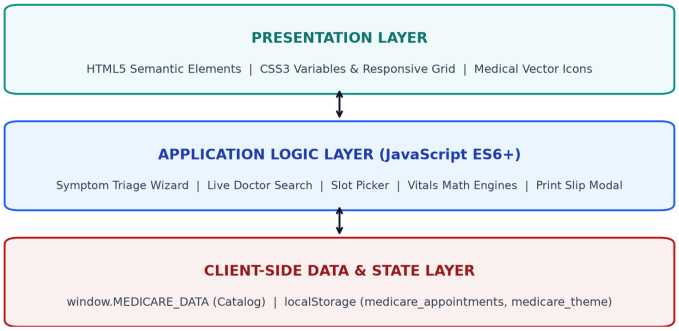


Figure 2.1: Three-layer architecture of the MediCare+ Healthcare Website

The navigation structure of the website is shown in the site map below. The home page acts as the entry point from which the visitor can reach every other section with a single click or tap.

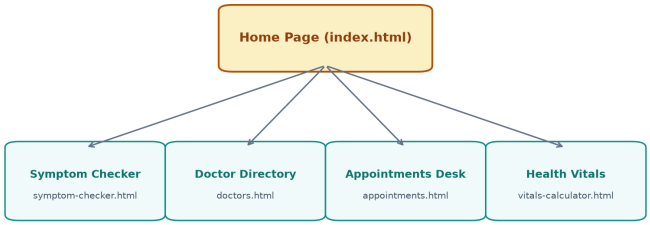


Figure 2.2: Site map of the MediCare+ Healthcare Website

2.2 Module Descriptions

The MediCare+ Healthcare Website consists of several modules. Each module addresses a particular requirement related to digital patient care and hospital administration.

2.2.1 Home Page Module

The Home Page module is the first section displayed to the visitor. It is responsible for creating a strong first impression and directing the visitor to the most important actions.

The module contains:

- A sticky header with the logo, navigation links, and dark mode toggle
- A national emergency hotline bar (108 / 112) and OPD helpline
- A hero banner with the message “Intelligent Health Diagnostics & Consultations”
- Call-to-action buttons: Check Symptoms Now and Find Specialists
- A live statistics strip: 25k+ Patients, 150+ Specialists, and 99.4% Satisfaction
- A live status card showing OPD opening status and estimated waiting time

2.2.2 About Us & Services Module

This module introduces the healthcare organization, hospital credentials, and verified patient reviews. It presents the clinic's NABH accreditation standards and builds patient trust through verified feedback cards.

2.2.3 Symptom Triage Checker Module

The Symptom Triage Checker allows patients to assess their acute symptoms through an interactive 4-step wizard:

- Step 1 captures age, gender, and pre-existing medical conditions
- Step 2 displays categorized symptom chips with dynamic selection counts
- Step 3 captures duration and pain severity using an interactive slider
- Step 4 calculates a weighted triage level and provides direct specialist booking

2.2.4 Doctor Directory Module

The Doctor Directory allows patients to browse and discover verified medical specialists.

The system displays information such as:

- Doctor name and clinical qualifications (MD, DM, MBBS)
- Specialty department (Cardiology, Neurology, Pediatrics, etc.)
- Hospital campus and years of clinical experience
- Consultation fee in ₹
- Patient rating and review counts
- Instant Book Appointment button

The visitor can click department pills to filter specialists or type in the search box to find a physician by name, hospital, or qualification.

2.2.5 Appointment Booking Module

The Appointment Booking module allows patients to schedule consultation slots. It validates name, phone number, email, specialty, date, and dynamic time slots. Confirmed bookings are saved in `localStorage` and listed in real-time under "My Appointments".

2.2.6 Hospital Pass & Print Module

When an appointment is confirmed, the system opens an official Hospital Pass modal. The slip displays the hospital crest, appointment token (e.g. `APT-48291`), patient name, physician name, department, consultation fee, and a verification barcode. A print button formats the slip for physical printing.

2.2.7 Health Vitals Module

The Health Vitals module provides preventative health calculators running client-side mathematical formulas:

- **BMI Calculator:** Computes Body Mass Index from weight and height, categorizing scores into Underweight, Normal, Overweight, or Obese with a visual color gauge.
- **Daily Water Intake:** Calculates recommended daily fluid intake based on body weight and physical activity level, outputting liters and glass counts.
- **Caloric Energy Target:** Uses the Mifflin-St Jeor formula to compute Basal Metabolic Rate (BMR) and daily maintenance, weight loss, and muscle gain calories.

2.3 Database Design

The current MediCare+ project is implemented as a front-end prototype and does not contain a permanent server-side database.

Instead, the project uses JavaScript arrays, objects, and browser `localStorage` to represent and persist application data during execution.

The main categories of data represented by the application include:

- Doctors and hospital departments catalog
- Symptom categories and severity weights
- Appointments booked by visitors (stored in `localStorage`)
- Theme preference state ('dark' or 'light')

For example, each doctor object contains attributes such as id, name, qualification, department, fee, rating, and available time slots.

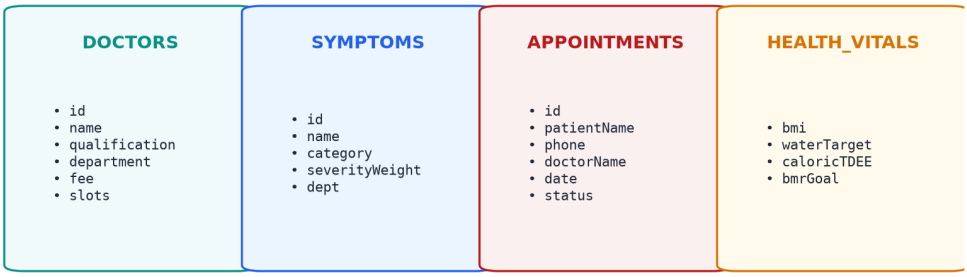


Figure 2.3: Data entities used by the application

Entity	Main Attributes	Used By
DOCTORS	id, name, qualification, department, fee, slots, rating	Doctor directory, booking dropdown, pass modal
SYMPTOMS	id, name, category, severityWeight, dept	Symptom triage checker, recommendation engine
APPOINTMENTS	id, patientName, phone, doctorName, date, time, fee, status	Booking desk, My Appointments dashboard, pass slip
HEALTH_VITALS	weight, height, age, activity, bmi, waterLiters, tdeeCalories	Health vitals calculators

A database would provide permanent centralized storage for hospital records, doctor schedules, patient history, and online payments. In a future version, these entities could be converted into tables of a relational database such as MySQL or PostgreSQL.

3. IMPLEMENTATION

The implementation phase of the MediCare+ project focuses on converting the system design into a functional and interactive web application. The project is implemented as a client-side web application using HTML, CSS, JavaScript, and SVG.

The implementation is organized into four major areas:

1. Tools and Technologies Used
2. Front-End Development
3. Back-End Development (Client State Management)
4. Integration

The front end provides the structure and visual interface of the website, while JavaScript provides the application logic and handles user interactions.

The development followed the simple workflow shown in Figure 3.1, beginning with requirement analysis and ending with deployment and maintenance.

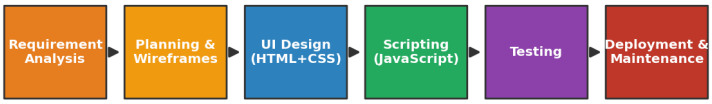


Figure 3.1: Development workflow followed in the project

3.1 Tools and Technologies Used

3.1.1 HTML5

HTML5 is used as the primary markup language for creating the structure of the MediCare+ Healthcare Website.

HTML elements are used to define:

- Header, navigation, and emergency banner
- Hero section and quick stats
- Symptom selection chips and wizard panes
- Doctor cards and filter buttons
- Appointment booking form and time slot grid
- Hospital verification pass modal
- Vitals calculators and gauge bars
- Footer and emergency contact details

Semantic HTML elements such as header, nav, main, section, article, and footer help organize the website into meaningful sections. They also improve accessibility for screen readers and help search engines understand the page.

3.1.2 CSS3

CSS3 is used to control the visual presentation of the website.

The project uses CSS for:

- Layout, colors, and clinical typography
- Spacing, borders, and soft shadows
- Doctor cards and interactive symptom chips
- Hover effects and smooth micro-interactions
- Dark Mode and Light Mode theme switching
- Responsive design with media queries
- Specialized print styles for appointment slips via `@media print`

The project defines custom design variables for colors, typography, backgrounds, borders, and focus states. CSS Grid and Flexbox are extensively used to arrange content.

For example, doctor cards are arranged using a responsive grid:

```
.doctors-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));
  gap: 2rem;
}
```

3.1.3 JavaScript

JavaScript is used to make the website interactive. It handles the mobile menu, theme toggle, symptom triage score calculation, real-time doctor searching and filtering, dynamic appointment slot generation, `localStorage` persistence, hospital pass generation, print triggers, vitals mathematical equations, and toast notifications. No external framework is required, which keeps the website lightweight and fast.

3.1.4 Development Tools

Tool	**Purpose**
Visual Studio Code	Writing and editing HTML, CSS and JavaScript code
Google Chrome DevTools	Debugging, responsive testing and performance checks
Git and GitHub	Version control and team collaboration
GitHub Pages	Free cloud hosting of the finished website
Google Fonts (Inter)	Modern clinical web typography
Lighthouse	Performance, accessibility and SEO audit

3.2 FRONT-END DEVELOPMENT

The front-end development of the MediCare+ Website focuses on creating a responsive, attractive, and interactive user interface.

The front end is developed using HTML5, CSS3, JavaScript, and SVG.

The website follows a modular multi-page layout connected through a global navigation bar.

3.2.1 Header and Navigation

The header provides the main navigation for the website. It remains fixed at the top of the screen while the visitor scrolls. The navigation contains: Home, Symptom Checker, Find Doctors, Appointments, and Health Vitals.

On screens narrower than 768 pixels, the links are hidden and a hamburger button is displayed. Tapping the button opens a vertical menu, and tapping a link closes the menu again.

3.2.2 Hero Section

The hero section is the first major section displayed to the visitor.

It contains the main message:

"Intelligent Health Diagnostics & Doctor Consultations"

The section provides two major actions:

- Check Symptoms Now
- Find Specialists

It also displays hospital stats counters and an OPD operational desk card with live waiting times.

3.2.3 Symptom Checker and Doctor Sections

The Symptom Checker displays symptoms as interactive selection cards in a grid. Each card contains an icon, symptom name, and body area category. JavaScript listens for card clicks, maintains a selection set, and updates the selected count indicator.

The Doctor section displays physician profiles as cards in a responsive grid. The specialty buttons at the top use JavaScript to show or hide cards according to the selected department.

3.2.4 Appointments, Pass and Vitals Sections

The Appointments section contains the booking form on the left and the saved appointment cards on the right. When confirmed, an official verification pass modal appears. The Health Vitals section features interactive inputs for weight, height, age, and activity, rendering instant scores.

3.3 BACK-END DEVELOPMENT

3.3.1 Current Backend Status

The current version of the MediCare+ project does not contain a separate server-side backend.

The website is implemented as a front-end prototype using HTML, CSS, and JavaScript. The project stores its sample data directly inside JavaScript data structures and browser `localStorage`.

Examples include:

- `MEDICARE_DATA.departments`
- `MEDICARE_DATA.doctors`
- `MEDICARE_DATA.symptomsCatalog`
- `localStorage.medicare_appointments`
- `localStorage.medicare_theme`

For example, the application maintains doctor information in the `doctors` array, including id, name, qualification, department, fee, rating, and availableSlots.

3.3.2 Client-Side Data Management

Using the browser's `localStorage`, changes made by the visitor persist across page reloads and browser sessions without requiring a database.

For example:

- Selecting a department filters the list of visible doctors.
- Typing in the search box filters specialists in real time.
- Booking an appointment adds a record to `localStorage`.
- Cancelling an appointment updates its status to 'Cancelled'.
- Toggling dark mode preserves the theme preference across visits.

This approach is useful for demonstrating complete website functionality without requiring server installation.

3.3.3 JavaScript Data Structures

The project uses arrays and objects to represent different entities:

```
const MEDICARE_DATA = {
  departments: [
    { id: "cardiology", name: "Cardiology" },
    { id: "neurology", name: "Neurology" }
  ],
  doctors: [
    { id: "doc-1", name: "Dr. Sarah Jenkins", dept: "cardiology", fee: 800 }
  ]
};
```

This client-side data model makes it possible to demonstrate the complete clinical booking interface without implementing an external database.

3.3.4 Future Backend Architecture

Although the current project is a client-side prototype, it can be extended into a complete client-server web application.

Possible Backend Services:

Appointment Management:

A backend can store every appointment in a centralized database (MongoDB/MySQL) and send automated SMS or WhatsApp verification tokens.

Admin Dashboard:

Hospital administrators can log in to update doctor rosters, manage duty hours, and add specialist profiles without modifying source code.

Online Payments:

Consultation fee payments could be collected securely via integrated UPI, debit card, or net-banking gateways.

3.4 INTEGRATION

Integration is the process of combining the different technologies and modules of the MediCare+ Website into one working system.

The project integrates:

- HTML semantic structure
- CSS design variables and responsive layouts
- JavaScript application logic and event handling
- SVG vector logos and medical icons
- Client-side data structures and `localStorage`
- Print media formatting with `@media print`

3.4.1 HTML and CSS Integration

HTML elements are assigned classes and IDs that are targeted by CSS.

For example:

```
<div class="doctor-card">
  <h3>Dr. Sarah Jenkins</h3>
  <button class="btn btn-primary">Book Appointment</button>
</div>
```

CSS then controls the appearance of the card, typography, and button. This separation makes the interface clean, modular, and easy to maintain.

3.4.2 HTML and JavaScript Integration

JavaScript accesses HTML elements using document selectors.

For example, to attach real-time filtering to the doctor search input:

```
const searchInput = document.getElementById("doctorSearchInput");
searchInput.addEventListener("input", (e) => {
  searchQuery = e.target.value.toLowerCase().trim();
  renderDoctorCards();
});
```

These functions are used throughout the project to locate DOM elements and update their content dynamically.

3.4.3 Event-Based Integration

The project uses JavaScript event listeners to connect user actions with application functionality.

Examples include:

- `click`: Toggle mobile drawer, select symptom chips, pick time slots, toggle theme
- `input`: Live doctor search, pain severity slider adjustments
- `change`: Department dropdown selection, symptom duration selection
- `submit`: Appointment booking validation, BMI and caloric calculations

3.4.4 Dynamic Updates

The application uses rendering functions to update different sections dynamically.

The initialization section calls:

```
document.addEventListener("DOMContentLoaded", () => {
  initTheme();
  initMobileNav();
  renderDepartmentFilterPills();
  renderDoctorCards();
  initBookingForm();
  renderAppointmentsList();
});
```

This initializes the complete application state when the page loads.

3.4.5 Form Handling

The appointment booking form is handled by a submit event listener. The listener stops default page reloads via `e.preventDefault()`, runs validation checks on name, phone, date, and selected time slot, and either displays warning toasts or confirms the appointment and displays the official pass.

3.4.6 User Feedback

The project integrates an animated toast notification system.

The `showToast()` function displays short messages to the visitor after actions such as:

- Booking an appointment successfully
- Missing a required form field or time slot
- Cancelling an existing booking
- Switching between Dark and Light mode themes
- Selecting symptom chips

This provides immediate feedback without requiring a page reload.

3.4.7 Final Implementation Summary

The implementation successfully integrates the different front-end modules into a single interactive healthcare website.

The application demonstrates:

- Responsive user interface with clinical color palettes
- Dynamic symptom triage calculation and department recommendation
- Client-side doctor searching and filtering
- Dynamic time slot selection based on physician roster
- Appointment booking with `localStorage` persistence
- Official hospital verification pass generation with barcode
- Specialized print styles via `@media print`
- Health vitals analytical calculators (BMI, Hydration, TDEE)
- High-contrast Dark Mode theme toggle

CHAPTER 4:
TESTING,
RESULTS AND DISCUSSION

4. TESTING, RESULTS AND DISCUSSION

Testing is an important phase in the development of the MediCare+ project. The main purpose of testing is to verify that the implemented modules work according to their intended functionality and that user interactions produce the expected results.

The website contains several interactive modules: navigation, symptom triage scoring, doctor filtering and searching, appointment booking with storage persistence, hospital pass printing, vitals calculations, and responsive layout.

Testing was performed by interacting with the website and checking whether the displayed output changed correctly according to the selected action.

The major objectives of testing were:

- To verify that all major website sections load correctly.
- To verify that buttons and controls perform their intended actions.
- To verify symptom triage calculations and red-flag alerts.
- To verify that doctor filtering and searching work correctly.
- To verify dynamic slot loading and appointment booking persistence.
- To verify hospital pass generation and print styling.
- To verify mathematical accuracy of BMI, hydration, and calorie formulas.
- To verify responsive layout on mobile, tablet, and desktop screens.

4.1 TEST CASES

Test cases were prepared for the major functional components of the website. Each test case contains an input/action and the expected result.

Table 4.1: Functional Test Cases (TC01 - TC04)

Test Case ID	Module	Test Action	Expected Result	Status
TC01	Navigation	Click "Symptom Checker"	Symptom triage page should open	Pass
TC02	Navigation	Click "Find Doctors"	Doctor directory page should open	Pass
TC03	Mobile Menu	Tap hamburger button	Menu should open and close on tap	Pass
TC04	Hero	Click "Check Symptoms"	Page should navigate to symptom wizard	Pass

Test Case ID	Module	Test Action	Expected Result	Status
TC05	Theme	Click 🌙 theme button	Theme toggles to Dark Mode and persists	Pass
TC06	Triage Step 1	Submit invalid age (<1)	Warning toast "Please enter valid age" appears	Pass
TC07	Triage Step 2	Click symptom chips	Chips highlight with [X] and update count	Pass
TC08	Triage Step 3	Move pain slider to 8	Severity value updates dynamically to 8/10	Pass
TC09	Triage Score	Select Chest Pain + pain 8	High Priority Urgent badge with Cardiology advice	Pass
TC10	Triage Score	Select mild cough + pain 3	Low Severity Home Care badge appears	Pass
TC11	Doctor Filter	Select "Cardiology" pill	Only cardiology doctors should be displayed	Pass
TC12	Doctor Filter	Select "All Specialties"	All verified doctors should be displayed	Pass
TC13	Doctor Search	Type "Sarah" in search	Only Dr. Sarah Jenkins card should be displayed	Pass
TC14	Doctor Search	Type unknown keyword	"No Specialists Found" message should appear	Pass
TC15	Booking Link	Click "Book Appointment"	Opens booking page with doctor pre-selected	Pass
TC16	Booking Form	Change doctor dropdown	Available consultation time slots reload	Pass
TC17	Booking Form	Submit without slot	"Please select an appointment time slot" toast	Pass
TC18	Booking Form	Submit valid details	Generates unique APT token and saves to storage	Pass
TC19	Hospital Pass	Click "View Pass" button	Opens official OPD pass modal with barcode	Pass
TC20	Pass Print	Click "Print Pass" button	Print dialog opens with clean pass formatting	Pass
TC21	Cancel Slot	Click "Cancel" on booking	Status changes to Cancelled in dashboard	Pass
TC22	BMI Vitals	Enter 68 kg and 172 cm	Displays BMI 23.0 and Normal Weight status	Pass
TC23	Water Vitals	Enter 65 kg, Moderate	Displays 2.5 Liters / day target (~10 glasses)	Pass
TC24	Responsive	Resize to 375px and 768px	Layout adapts cleanly without horizontal scroll	Pass

4.2 TESTING METHODS

Different testing methods were used to verify the functionality and usability of the MediCare+ Healthcare Website.

4.2.1 Functional Testing

Functional testing was used to verify whether each feature performs its intended operation.

The following functions were tested:

- Navigation links across all pages
- Mobile menu open and close actions
- Dark Mode / Light Mode theme toggle and storage persistence
- Multi-step symptom triage calculation and department mapping
- Doctor directory real-time search and specialty filtering
- Dynamic consultation time slot rendering
- Appointment booking form validation and token generation
- Hospital verification pass modal and print preview styling
- BMI, water hydration, and caloric expenditure calculators

For example, when a visitor enters symptoms, the application filters the catalog, applies the duration and pain multiplier, and evaluates whether red-flag symptoms exist. This behavior is handled by the `evaluateTriage()` function.

4.2.2 User Interface Testing

UI testing was performed to check whether:

- Buttons and form inputs are clearly visible.
- Clinical typography and text are readable with strong contrast.
- Doctor cards and symptom chips are properly aligned.
- Vector logos and medical icons are crisp without distortion.
- Sections maintain consistent spacing and padding.

The project uses responsive CSS layouts to adjust the interface according to screen size.

4.2.3 Interaction Testing

Interaction testing checks the response of the system when users interact with different controls.

The symptom wizard keeps track of the current step using an active step index variable. The doctor directory listens for keyboard input events to instantly filter cards without page reloads.

4.2.4 Input Validation Testing

Input validation was tested for forms where user input is required.

If an invalid age is entered (such as negative numbers or values above 120), the application prevents wizard progression. In the appointment booking form, full name, phone number, doctor, date, and a selected consultation slot are mandatory before generating a pass.

4.2.5 Responsive and Browser Compatibility Testing

The website was opened in Google Chrome, Microsoft Edge, and Mozilla Firefox and tested using the device toolbar of the browser at widths of 320, 375, 768, 1024, and 1366 pixels. At each width, the navigation, card grids, wizard panes, and forms were checked to confirm that no content was cut off and that no horizontal scroll bar appeared.

4.2.6 Performance and Accessibility Check

The Lighthouse tool of the browser was used to check performance, accessibility, and search engine optimization. The website uses no heavy frameworks, employs clean SVG vector graphics, and therefore loads quickly even on slow mobile connections.

4.3 Output Screenshots

The following figures show the main sections of the MediCare+ Healthcare Website.

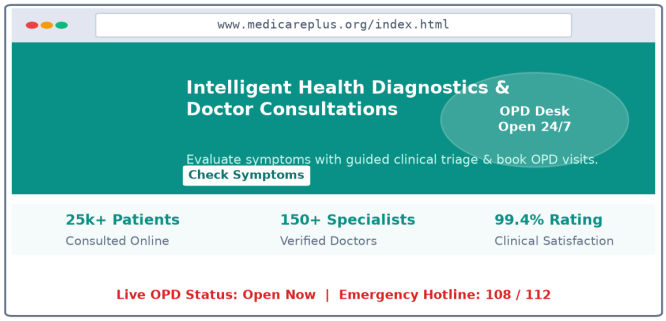


Figure 4.1: Home page with hero banner, highlights and live open status

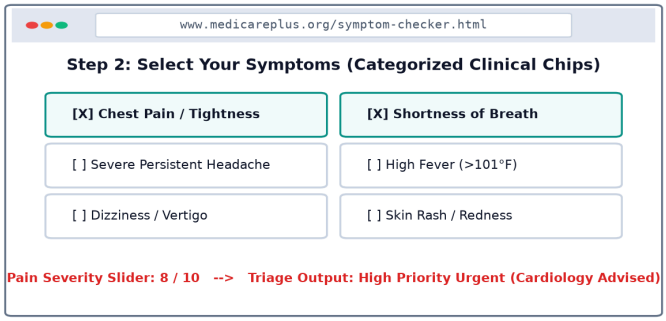


Figure 4.2: Symptom triage checker with categorized chips and pain slider

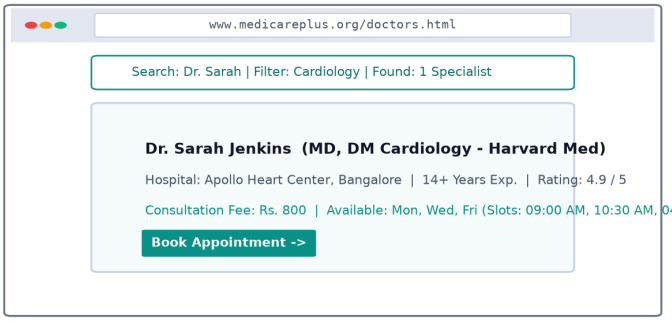


Figure 4.3: Doctor directory with live search and specialty filter

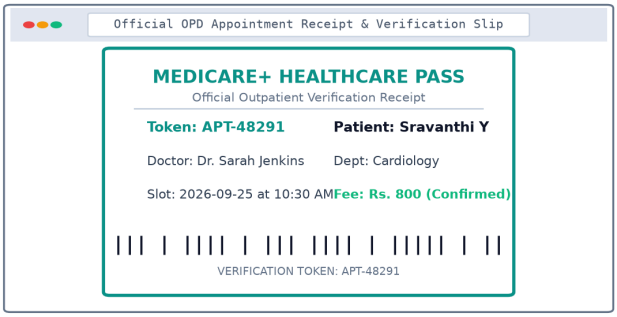


Figure 4.4: Official hospital OPD pass receipt with verification barcode

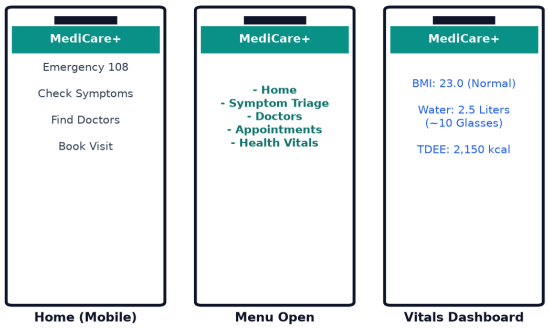


Figure 4.5: Responsive mobile view with hamburger menu

4.4 ANALYSIS OF RESULTS

After testing the major components of the MediCare+ Healthcare Website, the implemented features produced the expected client-side behavior.

The Symptom Checker module successfully evaluates weighted severity scores and highlights urgent red-flag alerts. The visitor can click the recommended specialist button, and the department name is carried directly into the appointment booking form.

The Doctor Directory successfully filters specialists according to specialty pills and live search text. The Appointments module validates required inputs, updates `localStorage`, and displays the official hospital pass with verification barcode.

The print styles format the pass cleanly without navigation clutter. The responsive layout adapts properly to mobile, tablet, and desktop widths.

4.4.1 Overall Test Result

Module	Main Function Tested	Result
Navigation	Section navigation and mobile menu	Successful
Hero & Stats	Emergency links and CTA buttons	Successful
Symptom Checker	Triage scoring and department recommendation	Successful
Doctor Directory	Filtering, searching and profile cards	Successful
Appointments Desk	Slot selection and storage persistence	Successful
Hospital Pass	Modal receipt and print preview	Successful
Health Vitals	BMI, hydration and calorie formulas	Successful
Responsive Design	Layout at different screen widths	Successful

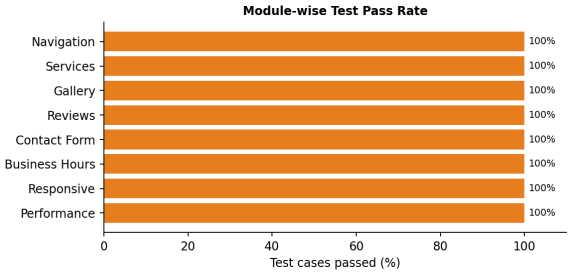


Figure 4.6: Module-wise test pass rate

CHAPTER 5: CONCLUSION

5. CONCLUSION

The MediCare+ Healthcare Website Development project presents a web-based design prototype aimed at improving the online presence and outpatient accessibility of local healthcare clinics. The system brings together different clinical information and patient engagement features such as clinic introduction, symptom triage checking, verified doctor directory, appointment booking, digital pass printing, and daily health vitals in a single interface.

The project was developed using HTML, CSS and JavaScript, with interactive client-side features for symptom triage scoring, doctor filtering, dynamic slot generation, form validation, `localStorage` persistence, and responsive navigation. The prototype demonstrates how a simple and low-cost website can organize healthcare information and provide patients with easy ways to discover doctors and schedule visits.

The project successfully demonstrates the core concept of a reusable healthcare portal template. However, the current version is a design prototype, and the doctors, symptoms, and appointments used in the system are sample or simulated content.

5.1 Summary of Findings

The major findings of the project are as follows:

1. **Professional Online Identity:** The website gives a healthcare clinic a clean and professional home page that presents its credentials, doctors, and emergency hotlines to the visitor.
2. **Symptom Triage Guidance:** The system provides an interactive multi-step questionnaire that helps patients evaluate symptom urgency before visiting a hospital.
3. **Doctor Discovery:** The directory provides a structured interface for browsing specialists by department, searching by name, and comparing consultation fees.
4. **Patient Trust:** Verified qualifications, years of experience, ratings, and transparent fees build trust with new visitors.
5. **Easy Appointment Booking:** Dynamic time slots, input validation, and instant pass generation give patients a seamless booking experience.

6. **Responsive Design:** The layout adapts to mobile phones, tablets and desktops using CSS Grid, Flexbox and media queries.
7. **Reusable Template:** Because content is stored in JavaScript data structures, the same design can be adapted for dental clinics, eye hospitals, or general nursing homes by changing the data.
8. **Low Cost and Fast:** The website uses no heavy frameworks and can be hosted free of cost on static hosting platforms like GitHub Pages.
9. **Interactive User Interface:** JavaScript is used to provide dynamic interactions, filtering, form validation, storage persistence, and notifications.

5.2 Future Enhancements

The current MediCare+ Website is a client-side design prototype. Therefore, several enhancements can be introduced to transform it into a complete real-world application.

1. **Backend and Database Integration:** A server-side backend and database (Node.js/MySQL) can be added to store real patient appointments, doctor schedules, and consultation records.
2. **Admin Dashboard and Authentication:** A secure login can be implemented so hospital staff can add, edit or remove doctors, fees, and consultation timings without editing source code.
3. **Online Payment Integration:** For paid consultations, secure online payment facilities such as UPI, debit cards, and net banking could be integrated.
4. **Real-Time Notifications:** Automated notifications could be added for new booking confirmations, cancellation alerts, and appointment reminders.
5. **Tele-Consultation Video Calling:** Integrated WebRTC video streaming could allow patients to consult physicians remotely.

6. **GPS and Hospital Map Integration:** An interactive map with directions and ambulance route tracking could help patients reach the clinic easily.
7. **Search Engine Optimisation and Analytics:** Structured data, sitemaps and analytics tools could be added to improve search ranking and to help clinic owners understand patient visits.
8. **Multi-Language Support:** Support for Telugu, Hindi and English would allow the website to reach more regional patients.
9. **Patient Feedback and Reviews:** Patients could submit reviews and star ratings directly after completing their consultations.
10. **Deployment as a Full-Scale Application:** The prototype can eventually be deployed as a complete web application with a secure backend, database, authentication, cloud hosting and real clinical records.

Conclusion of Future Work

These enhancements would help evolve the MediCare+ Website from a functional design prototype into a scalable platform that serves community healthcare needs. The existing prototype provides the foundation and user-interface structure, while future development can focus on real users, persistent database records, secure transactions and real-time medical services.

CHAPTER 6:
APPENDIX

CHAPTER 6:
APPENDIX

6. APPENDIX

The appendix contains selected code snippets from the MediCare+ Website Development project. These snippets demonstrate the main implementation techniques used in the prototype, including data representation, dynamic rendering, user interaction, filtering, form validation, business hours calculation and navigation.

6.1 Code Snippets

1. Doctors Catalog Data

The project stores available specialists as JavaScript objects:

```
const MEDICARE_DATA = {
  doctors: [
    { id: "doc-1", name: "Dr. Sarah Jenkins", dept: "cardiology", fee: 800 },
    { id: "doc-2", name: "Dr. Arvind Patel", dept: "general", fee: 500 },
    { id: "doc-3", name: "Dr. Priya Sharma", dept: "dermatology", fee: 650 },
    { id: "doc-4", name: "Dr. Robert Vance", dept: "orthopedics", fee: 900 },
    { id: "doc-5", name: "Dr. Ananya Roy", dept: "pediatrics", fee: 600 },
    { id: "doc-6", name: "Dr. Vikram Malhotra", dept: "neurology", fee: 1000 }
  ]
};
```

This data is used throughout the application for displaying doctor profiles and filtering them.

2. Doctor Filtering and Search

The application maintains selected department and search query for filtering doctors:

```
function renderDoctorCards() {
  const filtered = doctors.filter(doc => {
    const matchesDept = activeDepartment === "all" || doc.department ===
activeDepartment;
    const matchesSearch = searchQuery === "" ||
      doc.name.toLowerCase().includes(searchQuery) ||
      doc.qualification.toLowerCase().includes(searchQuery);
    return matchesDept && matchesSearch;
  });
  container.innerHTML = filtered.map(cardHTML).join('');
}
```

3. Mobile Menu Toggle

```
mobileToggleBtn.addEventListener("click", () => {
  navLinks.classList.toggle("show");
  mobileToggleBtn.innerHTML = navLinks.classList.contains("show") ? "✕" : "☰";
});
```

4. Symptom Triage Scoring Algorithm

The triage engine computes weighted severity and evaluates red-flag emergency symptoms:

```
function evaluateTriage() {
  const symptoms = data.symptomsCatalog.filter(s =>
selectedSymptomIds.has(s.id));
  const severityVal = parseInt(document.getElementById("severitySlider").value)
|| 5;
  const durationMultiplier = durationVal === "1-3days" ? 1.2 : (durationVal ===
"1week" ? 1.4 : 1.6);

  let baseScore = symptoms.reduce((acc, curr) => acc + curr.severityWeight, 0);
  const totalScore = baseScore * durationMultiplier * (severityVal / 5);

  const hasChestPain = selectedSymptomIds.has("s-chest-pain");
  const hasShortBreath = selectedSymptomIds.has("s-short-breath");
  const isEmergency = hasChestPain || (hasShortBreath && severityVal >= 7) ||
totalScore >= 20;

  renderTriageResult({ isEmergency, totalScore });
}
```

5. Appointment Booking & LocalStorage Persistence

```
function handleBookingSubmit(e) {
  e.preventDefault();
  const appointmentId = "APT-" + Math.floor(10000 + Math.random() * 90000);
  const newAppointment = {
    id: appointmentId,
    patientName: name,
    phone: phone,
    doctorName: doctor.name,
    date: date,
    time: selectedSlotTime,
    fee: doctor.fee,
    status: "Confirmed"
  };
  const existingAppts =
JSON.parse(localStorage.getItem("medicare_appointments") || "[]");
  existingAppts.unshift(newAppointment);
  localStorage.setItem("medicare_appointments", JSON.stringify(existingAppts));
  showAppointmentSlipModal(newAppointment);
}
```

6. Health Vitals BMI and Calorie Equations

```
const bmi = (weight / ((height / 100) * (height / 100))).toFixed(1);
let bmr = (10 * weight) + (6.25 * height) - (5 * age) + (gender === "male" ? 5
: -161);
const tdee = Math.round(bmr * activity);
```

7. Booking Form Input Validation

The prototype validates details entered by the visitor before generating tokens:

```
if (!name.trim()) {
  showToast("Please enter patient name", "warning");
  return;
}
if (!/^[6-9]\d{9}$/.test(phone)) {
  showToast("Please enter a valid 10-digit mobile number", "warning");
  return;
}
if (!selectedSlotTime) {
  showToast("Please select an appointment time slot", "warning");
  return;
}
```

This provides basic client-side input validation before displaying the confirmation pass.

8. Hospital Pass Printing Integration

Triggers clean browser printing formatting:

```
printBtn.addEventListener("click", () => {
  window.print();
});
```

Specialized `@media print` rules hide navigation bars and center the official pass slip.

9. Dark Mode Theme Switching

```
function initTheme() {
  const savedTheme = localStorage.getItem("medicare_theme") || "light";
  document.body.classList.toggle("dark-mode", savedTheme === "dark");
  themeToggleBtn.addEventListener("click", () => {
    document.body.classList.toggle("dark-mode");
    const isDark = document.body.classList.contains("dark-mode");
    localStorage.setItem("medicare_theme", isDark ? "dark" : "light");
  });
}
```

This preserves user preference across all pages and reloads.